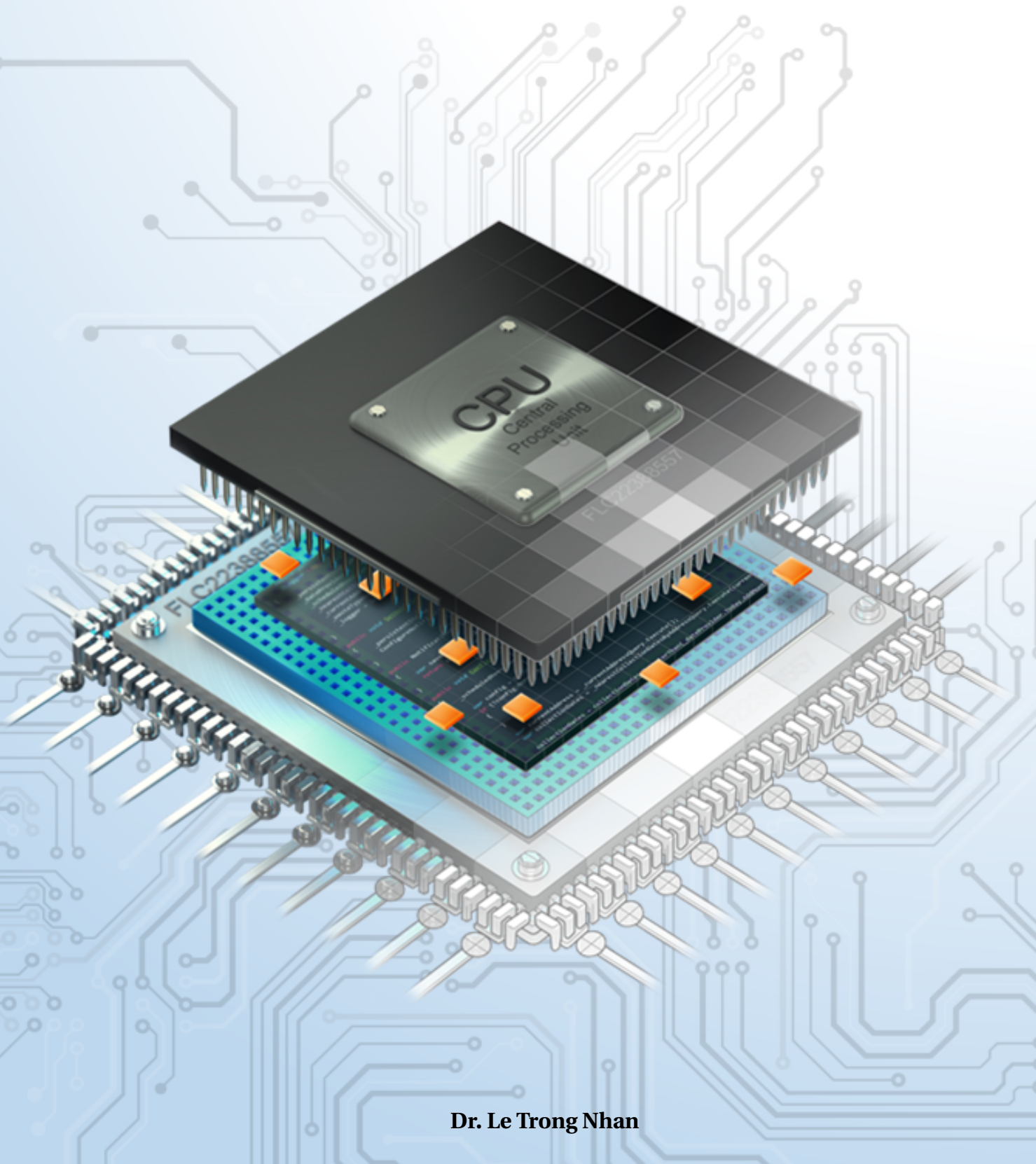




HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
COMPUTER ENGINEERING

Microcontroller



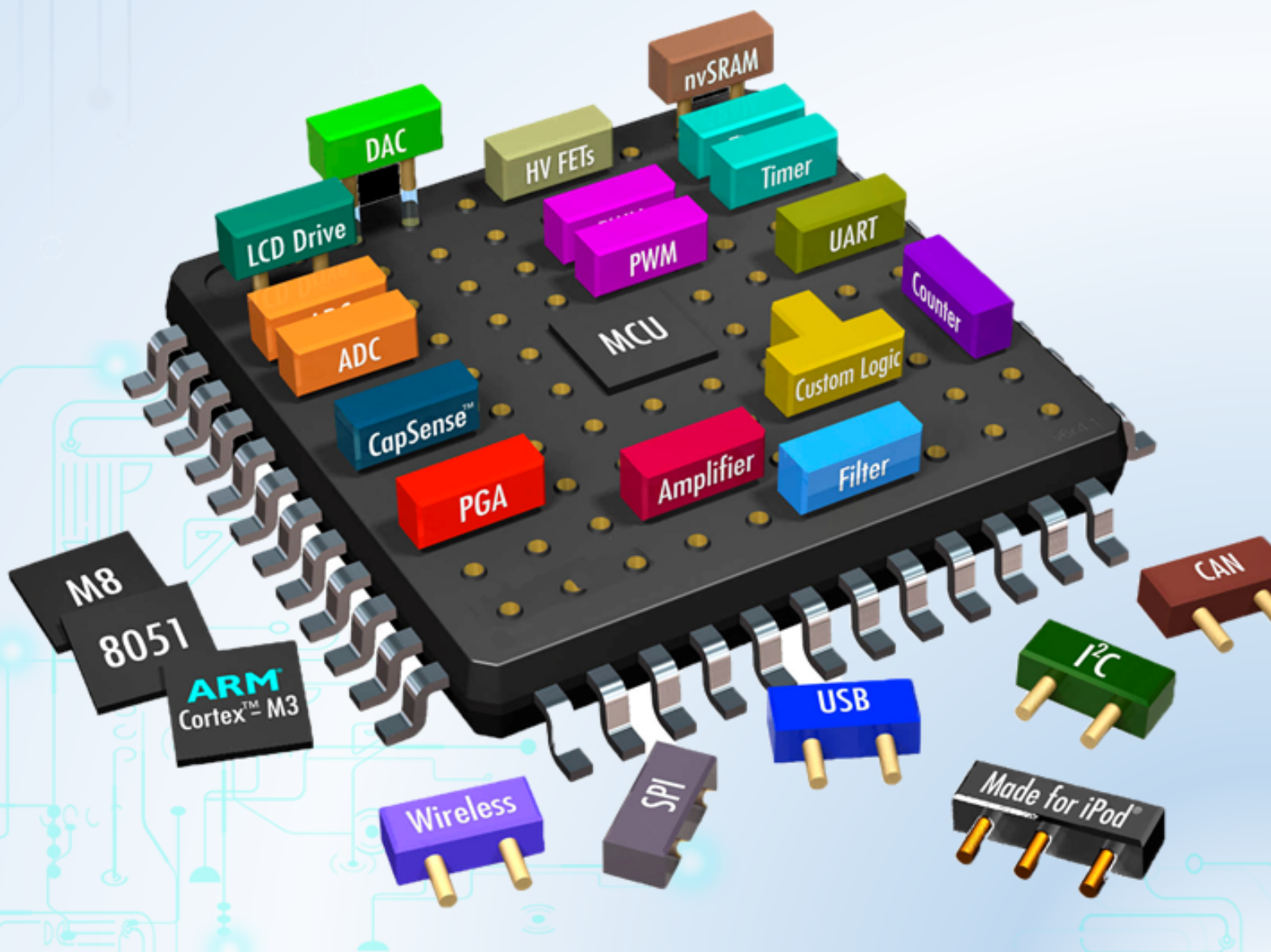
Dr. Le Trong Nhan

Mục lục

Chapter 1. LED Animations	7
1 Introduction	8
2 First project on STM32Cube	9
3 Simulation on Proteus	15
4 Exercise and Report	21
4.1 Exercise 1	21
4.2 Exercise 2	22
4.3 Exercise 3	25
4.4 Exercise 4	25
4.5 Exercise 5	30
4.6 Exercise 6	35
4.7 Exercise 7	36
4.8 Exercise 8	37
4.9 Exercise 9	37
4.10 Exercise 10	37

CHƯƠNG 1

LED Animations



1 Introduction

In this manual, the STM32CubeIDE is used as an editor to program the ARM microcontroller. STM32CubeIDE is an advanced C/C++ development platform with peripheral configuration, code generation, code compilation, and debug features for STM32 microcontrollers and microprocessors.



Hình 1.1: STM32Cube IDE for STM32 Programming

The most interest of STM32CubeIDE is that after the selection of an empty STM32 MCU or MPU, or preconfigured microcontroller or microprocessor from the selection of a board, the initialization code generated automatically. At any time during the development, the user can return to the initialization and configuration of the peripherals or middleware and regenerate the initialization code with no impact on the user code. This feature can simplify the initialization process and speedup the development application running on STM32 micro-controller. The software can be downloaded from the link bellow:

https://ubc.sgp1.digitaloceanspaces.com/BKU_Softwares/STM32/stm32cubeide_1.7.0.zip

Moreover, for a hangout class, the program is firstly simulated on Proteus. Students are also supposed to download and install this software as well:

https://ubc.sgp1.digitaloceanspaces.com/BKU_Softwares/STM32/Proteus_8.10_SP0_Pro.exe

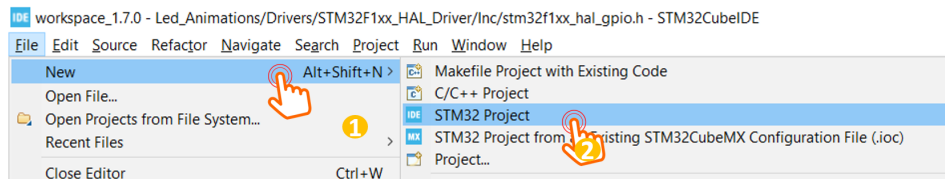
The rest of this manual consists of:

- Create a project on STM32Cube IDE
- Create a project on Proteus
- Simulate the project on Proteus

Finally, students are supposed to finish 10 different projects.

2 First project on STM32Cube

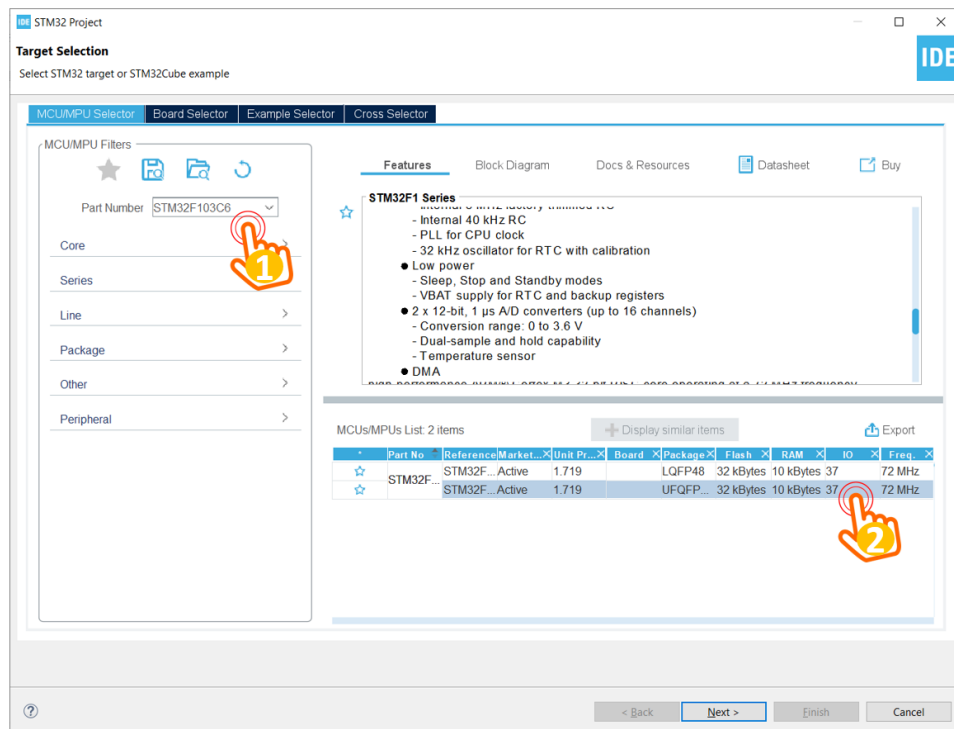
Step 1: Launch STM32CubeIDE, from the menu **File**, select **New**, then chose **STM32 Project**



Hình 1.2: Create a new project on STM32CubeIDE

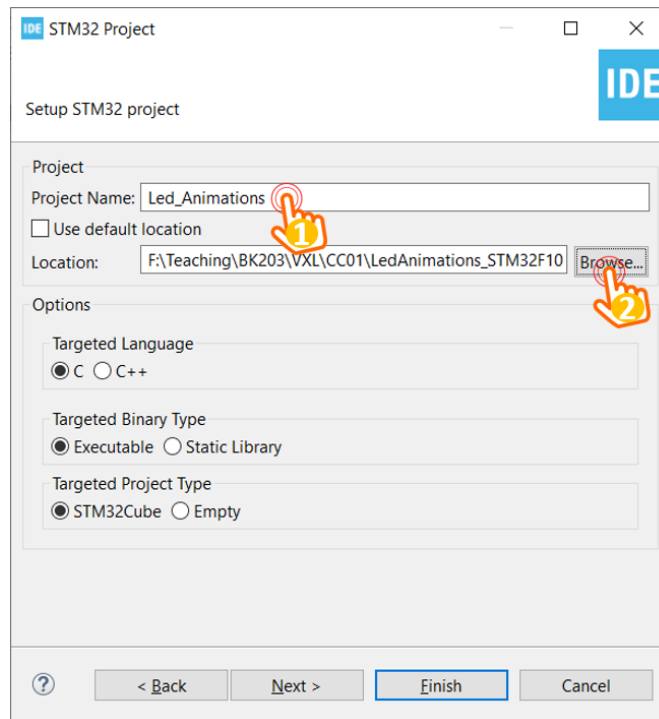
The IDE needs to download some packages, which normally takes time in this first time a project is created.

Step 2: Select the STM32F103C6 in the following dialog, then click on **Next**



Hình 1.3: Select the target device

Step 3: Provide the **Name** and the **Location** for the project.



Hình 1.4: Select the target device

It is important to notice that the **Targeted Project Type** should be **STM32Cube**. In the case this option is disabled, step 1 must be repeated. The location path should not contain special characters (e.g. the space). Finally, click on the **Next** button.

Step 4: On the last dialog, just keep the default firmware version and click on **Finish** button.

Step 5: The project is created and the wizard for configuration is displayed. This utility from CubeIDE can simplify the configuration process for an ARM microcontroller like the STM32.

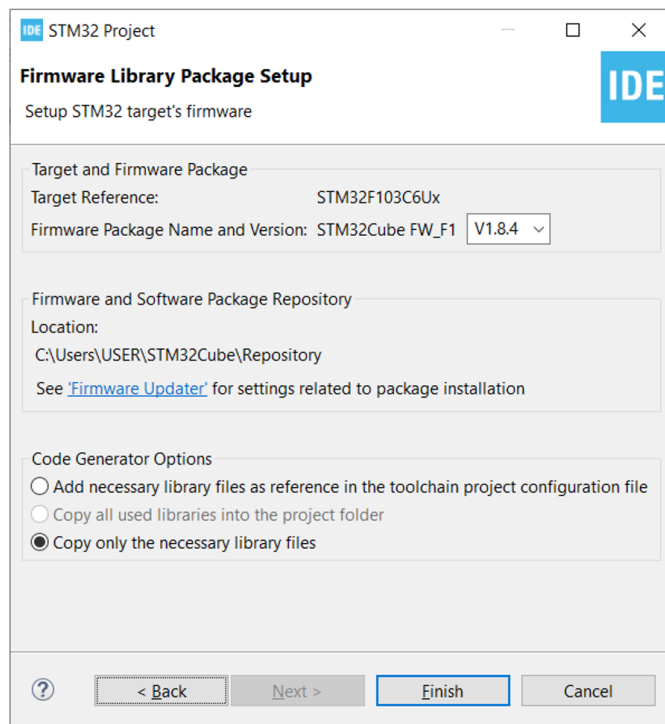
From the configuration windows, select **Pin configuration**, select the pin **PA5** and set to **GPIO Output** mode, since this pin is connected to an LED in the STM32 development kit.

Step 6: Right click on PA5 and select **Enter user label**, and provide the name for this pin (e.g. **LED_RED**). This step helps programming afterward more memorable.

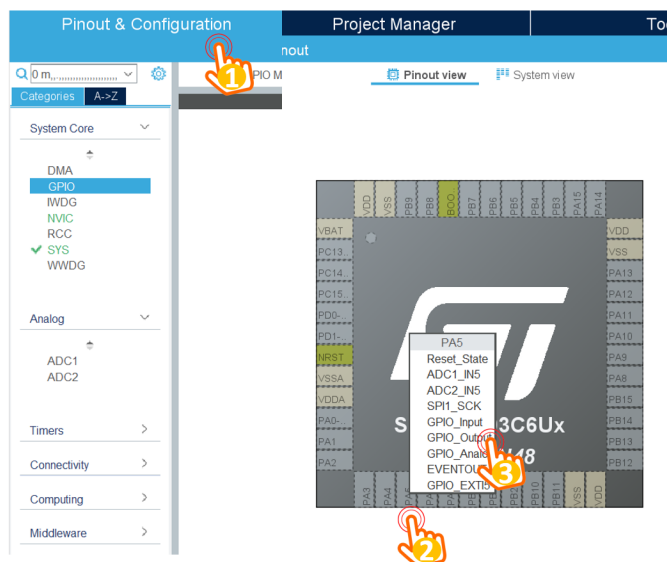
Finally, save the configuration process by pressing **Ctrl + S** and confirm this step by clicking on **OK** button. The code generation is started.

Step 7: Implement the first blinky project in the main function as follow:

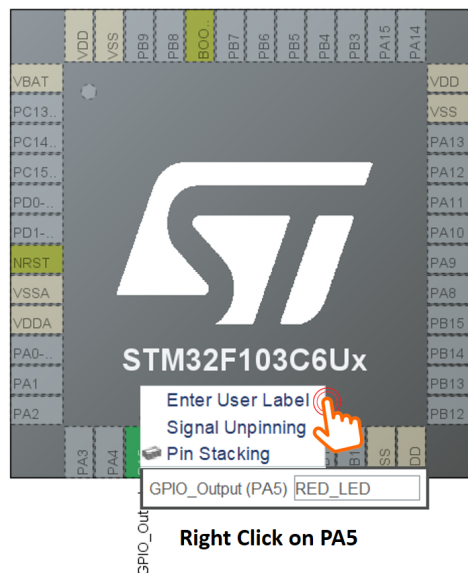
```
1 int main(void)
2 {
3     /* USER CODE BEGIN 1 */
4
5     /* USER CODE END 1 */
```



Hình 1.5: Keep default firmware version



Hình 1.6: Set PA5 to GPIO Output mode



Hình 1.7: Provide a name for PA5

6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33

```

/* MCU Configuration
-----
*/

/* Reset of all peripherals, Initializes the Flash
interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
/* USER CODE BEGIN 2 */

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */

while (1)
{

```

```

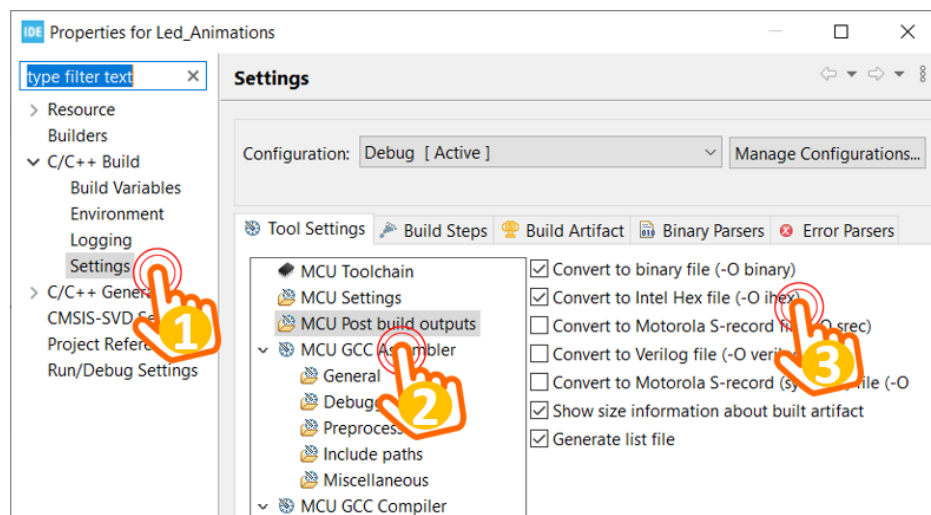
34     HAL_GPIO_TogglePin(LED_RED_GPIO_Port , LED_RED_Pin);
35     HAL_Delay(1000);
36     /* USER CODE END WHILE */
37
38     /* USER CODE BEGIN 3 */
39 }
40 /* USER CODE END 3 */
41 }

```

Program 1.1: First blinky LED project

Actually, what is added to the main function is line number 34 and 35. Please put your code in a right place, otherwise it can be deleted when the code is generated (e.g. change the configuration of the project). When coding, frequently use the suggestions by pressing **Ctrl+Space**.

Step 7: Due to the simulation on Proteus, the hex file should be generated from STM32Cube IDE. From menu **Project**, select **Properties** to open the dialog bellow:



Hình 1.8: Config for hex file output

Navigate to **C/C++ Build**, select **Settings**, **MCU Post build outputs**, and check to the **Intel Hex file**.

Step 8: Build the project by clicking on menu **Project** and select **Build Project**. Please check on the output console of the IDE to be sure that the hex file is generated, as follow:

```

22:36:06 **** Incremental Build of configuration Debug for project Led_Animations ****
make -j8 all
arm-none-eabi-size  Led_Animations.elf
   text    data    bss     dec     hex filename
   4596     20    1572    6188    182c Led_Animations.elf
Finished building: default.size.stdout

22:36:06 Build Finished. 0 errors, 0 warnings. (took 272ms)

```

Hình 1.9: Compile the project and generate Hex file

The hex file is located under the **Debug** folder of your project, which is used for the simulation in Proteus afterward. In the case a development kit is connected to your PC, from menu **Run**, select **Run** to download the program to the hardware platform.

In the case there are multiple project in a work-space, double click on the project name to activate this project. Whenever a project is built, check the output files to make sure that you are working in a right project.

3 Simulation on Proteus

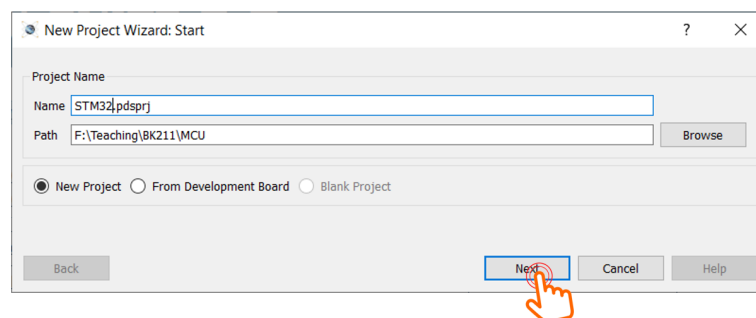
For an online training, a simulation on Proteus can be used. The details to create an STM32 project on Proteus are described bellow.

Step 1: Launch Proteus (with administration access) and from menu **File**, select **New Project**.



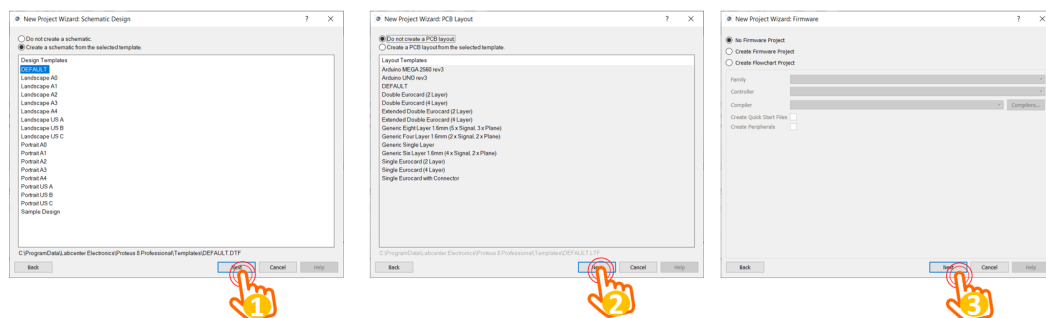
Hình 1.10: Create a new project on Proteus

Step 2: Provide the name and the location of the project, then click on **Next** button.



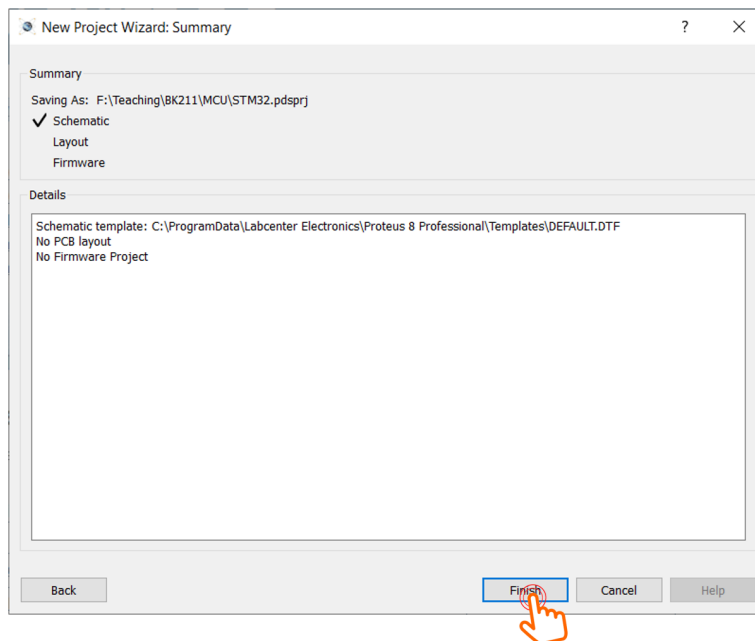
Hình 1.11: Provide project name and location

Step 3: For following dialog, just click on **Next** button as just a schematic is required for the lab.



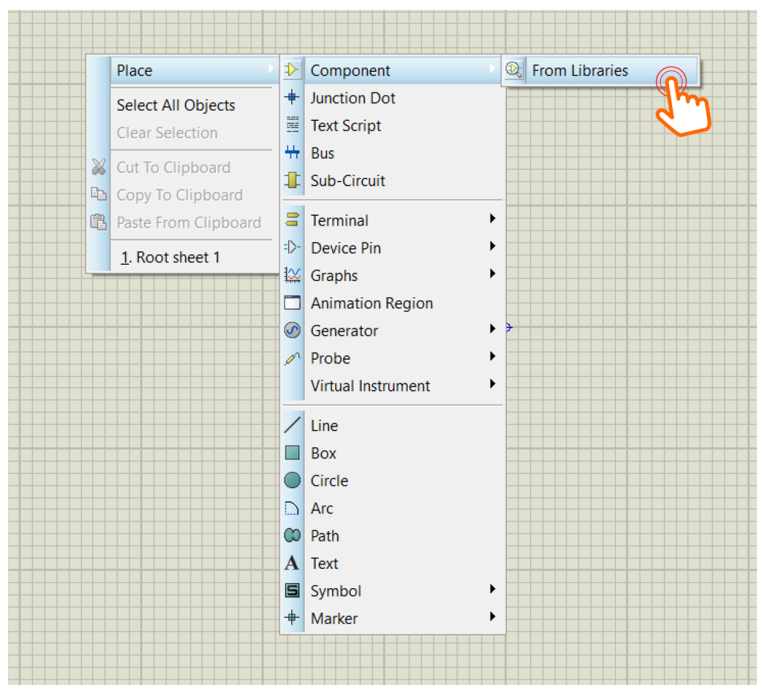
Hình 1.12: Keep the default options by clicking on Next

Step 4: Finally, click on **Finish** button to close the project wizard.



Hình 1.13: Finish the project wizard

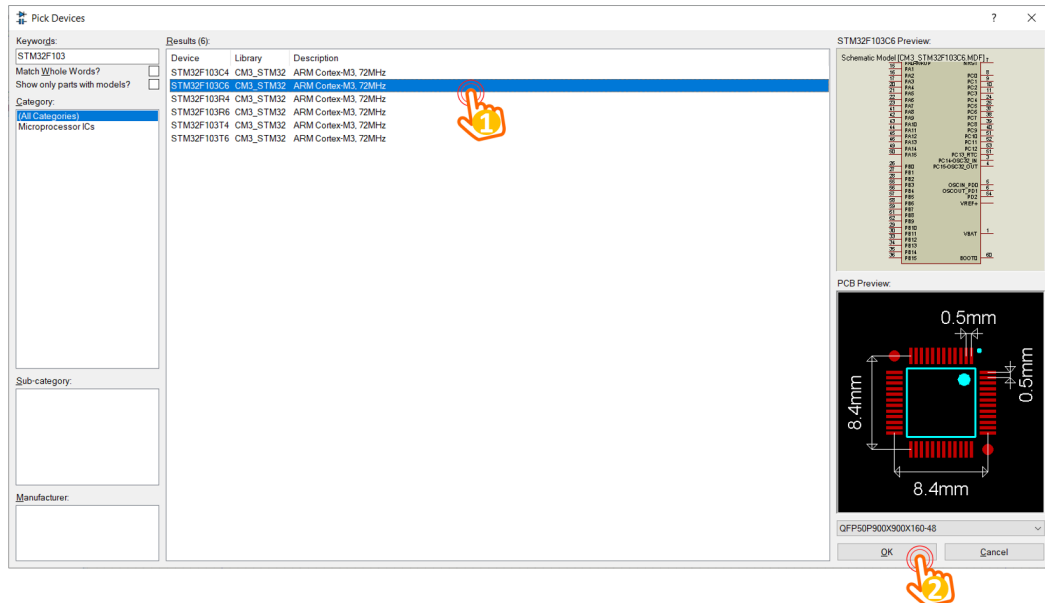
Step 5: On the main page of the project, right click to select **Place, Components, From Libraries**, as follows:



Hình 1.14: Select a component from the library

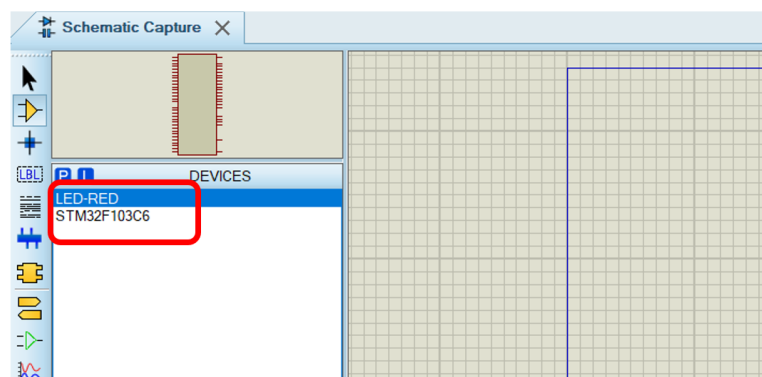
If there is an error with no library found, please restart the Proteus software with Run as administrator option.

Step 6: From the list of components in the library, select STM32F103C6, as follows:



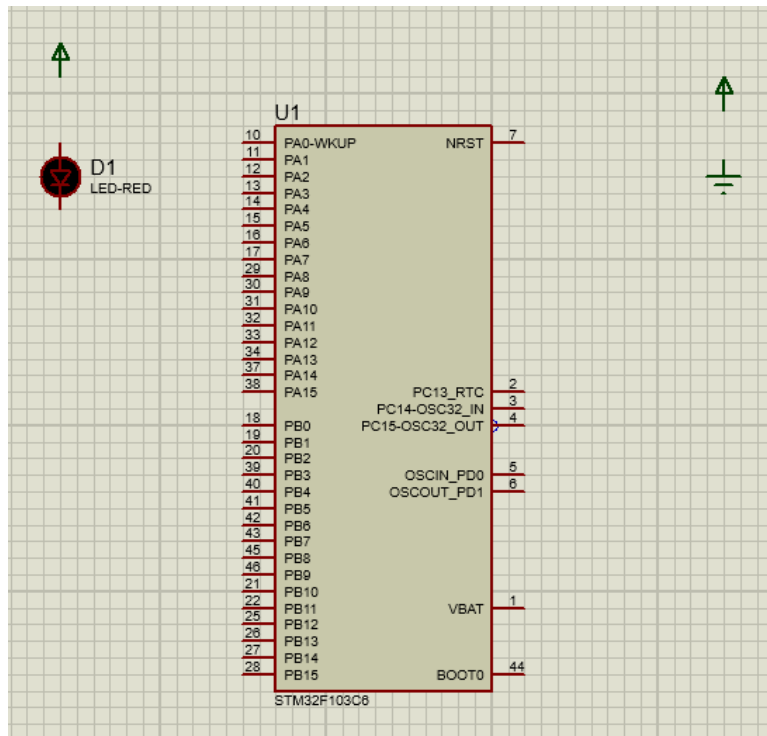
Hình 1.15: Select STM32F103C6

Repeat step 5 and 6 to select an LED, named **LED-RED** in Proteus. Finally, these components are appeared on the DEVICES windows, which is on left hand side as follows:



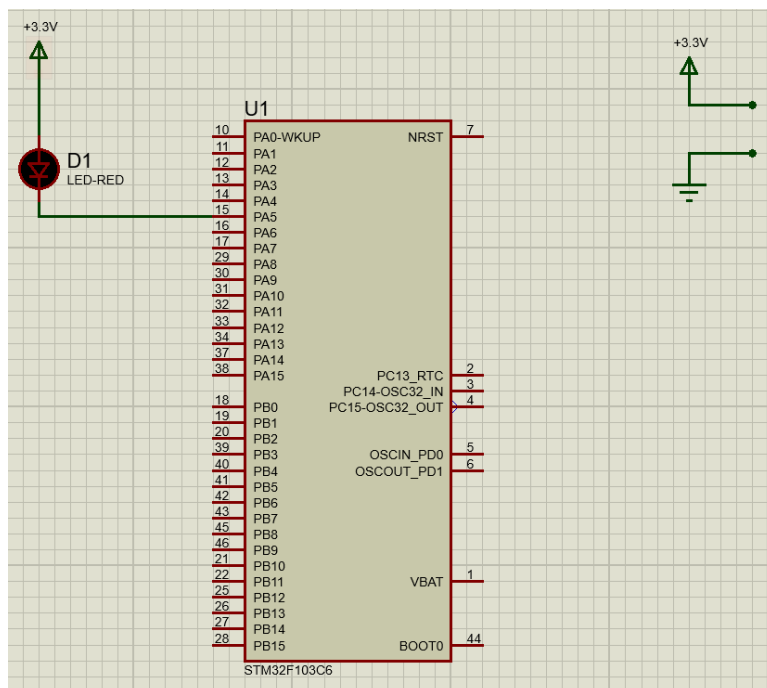
Hình 1.16: STM32 and an LED in the project

Step 7: Place the components to the project: right click on the main page, select on **Place, Component**, and select device added in Step 6. To add the Power and the Ground, right click on the main page, select on **Place, Terminal**. The result in this step is expected as follows:



Hình 1.17: Place components to the project

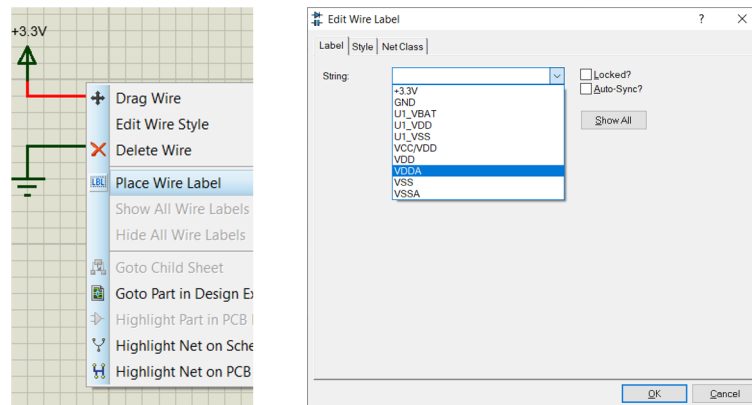
Step 8: Start wiring the circuit. The negative pin of the LED is connected to PA5 while its positive pin is connected to the power supply. For the power and the ground on the right, just make a short wire, which will be labeled in the next step.



Hình 1.18: Connect components and set the power to 3.3V

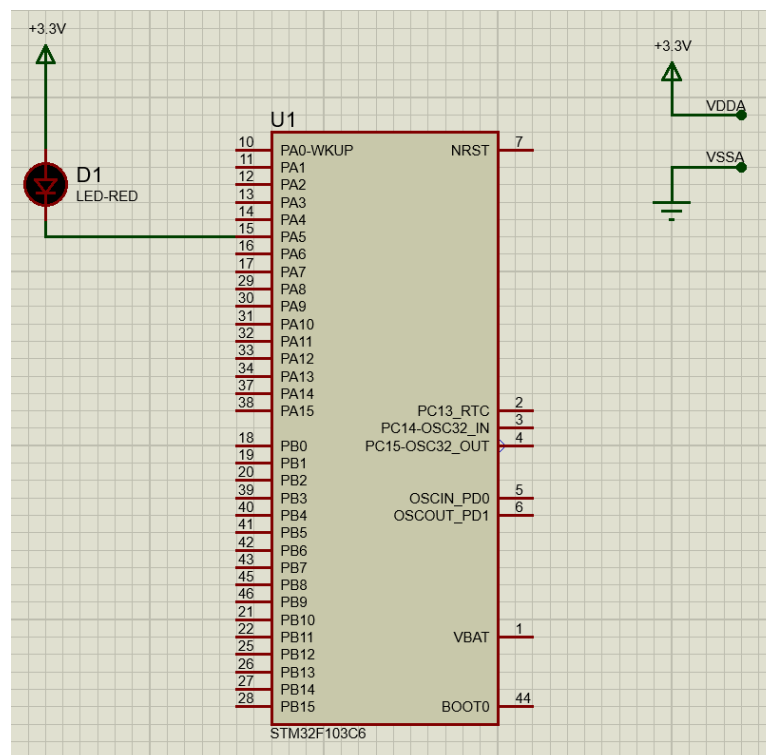
In this step, also double click on the power supply in order to provide the String property to **+3.3V**.

Step 8: Right click on the wire of the power supply and the ground, and select **Place wire Label**



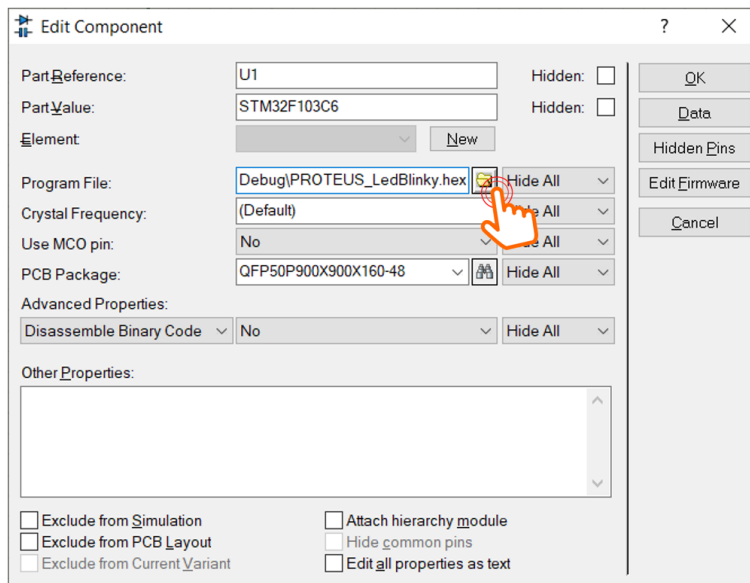
Hình 1.19: Place label for Power and Ground

This step is required as VDDA and VSSA of the STM32 must be connected to provide the reference voltage. Therefore, VDDA is connected to 3.3V, while the VSSA is connected to the Ground. Finally, the image of our schematic is shown bellow:



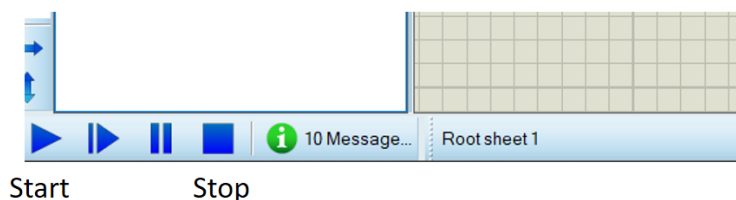
Hình 1.20: Finalize the schematic

Step 9: Double click on the STM32, and set the **Program File** to the Hex file, which is generated from Cube IDE, as following:



Hình 1.21: Set the program of the STM32 to the hex file from Cube IDE

From now, the simulation is ready to start by clicking on the menu **Debug**, and select on **Run simulation**. To stop the simulation, click on **Debug** and select **Stop VMS Debugging**. Moreover, there are some quick access bottom on the left corner of the Proteus to start or stop the simulation, as shown following:



Hình 1.22: Quick access buttons to start and stop the simulation

If everything is success, students can see the LED is blinking every second. Please stop the simulation before updating the project, either in Proteus or STM32Cube IDE. However, the step 9 (set the program file for STM32 in Proteus) is required to do once. Beside the toggle instruction, student can set or reset a pin as following:

```
1 while (1){
2     HAL_GPIO_WritePin(LED_RED_GPIO_Port , LED_RED_Pin ,
3       GPIO_PIN_SET);
4     HAL_Delay(1000);
5     HAL_GPIO_WritePin(LED_RED_GPIO_Port , LED_RED_Pin ,
6       GPIO_PIN_RESET);
7     HAL_Delay(1000);
8 }
```

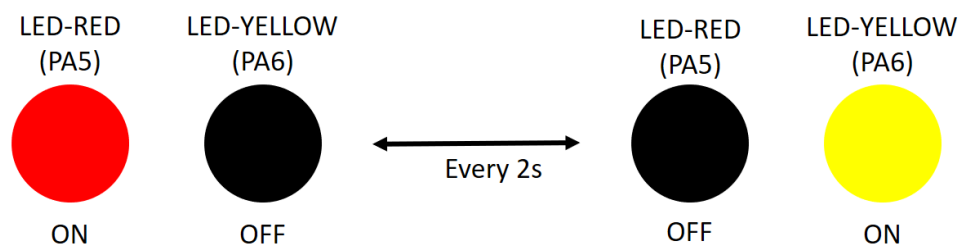
Program 1.2: An example for LED blinky

4 Exercise and Report

4.1 Exercise 1

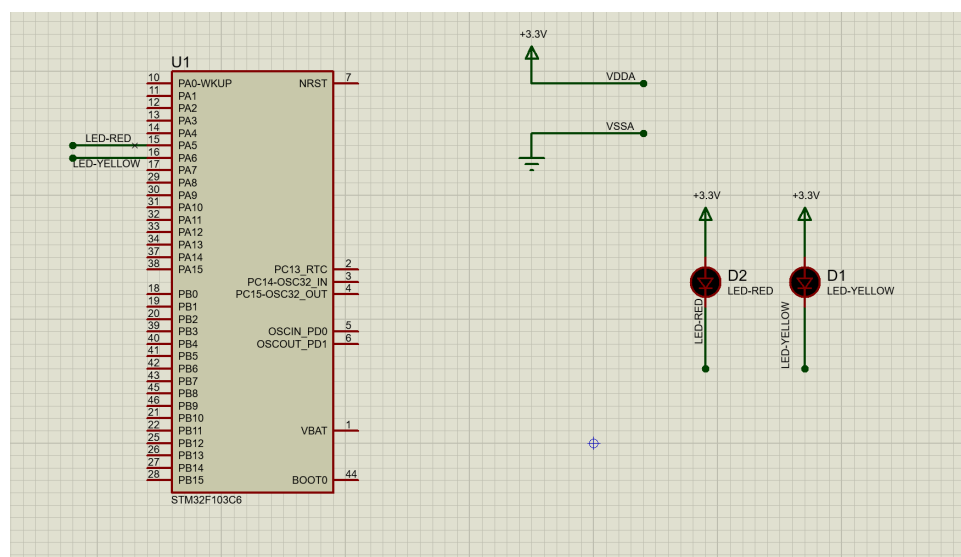
From the simulation on Proteus, one more LED is connected to pin **PA6** of the STM32 (negative pin of the LED is connected to PA6). The component suggested in this exercise is **LED-YELLOW**, which can be found from the device list.

In this exercise, the status of two LEDs are switched every 2 seconds, as demonstrated in the figure bellow.



Hình 1.23: State transitions for 2 LEDs

Report 1: Depict the schematic from Proteus simulation in this report. The caption of the figure is a downloadable link to the Proteus project file (e.g. a github link).



Hình 1.24: Link Github: <https://github.com/NguyenNhatTien-k23/STM32-Lab1>

Report 2: Present the source code in the infinite loop while of your project. If a user-defined functions is used, it is required to present in this part. A brief description can be added for this function (e.g. using comments). A template to present your source code is presented bellow.

```
1 while (1)
```

```

2  {
3      /* USER CODE END WHILE */
4
5      /* USER CODE BEGIN 3 */
6
7      //LED-RED on
8      if(state == 0){
9          HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, RESET
10         );
11         HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
12         SET);
13     }
14
15     //LED-YELLOW on
16     if(state == 1){
17         HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, SET);
18         HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
19         RESET);
20     }
21
22     --counter;
23     if(counter <= 0){
24         counter = 2;
25         state = !state;
26     }
27     HAL_Delay(1000);
28 }
29 /* USER CODE END 3 */

```

Program 1.3: Source code in the while loop

4.2 Exercise 2

Extend the first exercise to simulate the behavior of a traffic light. A third LED, named **LED-GREEN** is added to the system, which is connected to **PA7**. A cycle in this traffic light is 5 seconds for the RED, 2 seconds for the YELLOW and 3 seconds for the GREEN. The LED-GREEN is also controlled by its negative pin.

Similarly, the report in this exercise includes the schematic of your circuit and a your source code in the while loop.

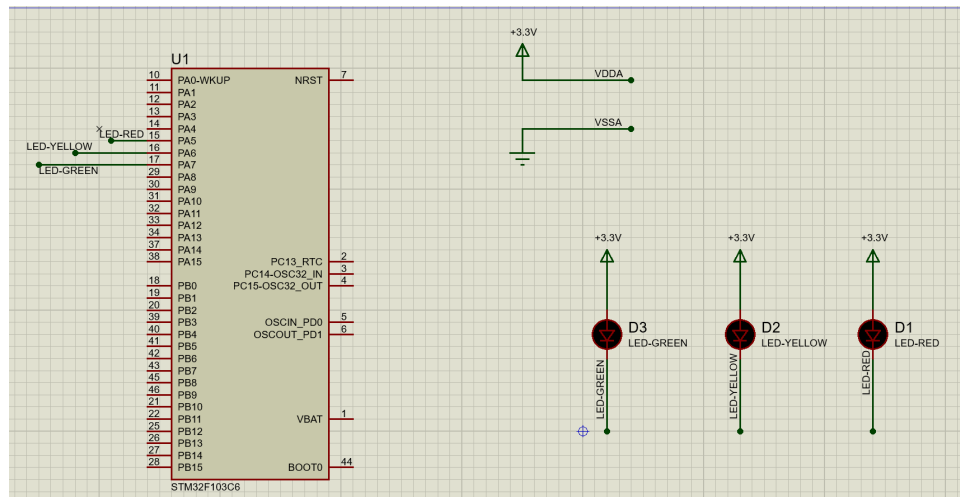
Report 1: Present the schematic.

Report 2: Present the source code in while.

```

1  while (1)
2  {
3      /* USER CODE END WHILE */
4

```



Hình 1.25: Schematic for Exercise 2

```

5      /* USER CODE BEGIN 3 */
6
7      if(led_counter <= 0){
8          led_state = GetNextState(led_state);
9          led_counter = GetStateCounter(led_state);
10         DrawLedState(led_state);
11     }
12     --led_counter;
13     HAL_Delay(1000);
14 }
15
16 void DrawLedState(int led_state){
17     switch(led_state){
18     case STATE_GREEN:
19         HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
20             RESET);
21         HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
22             SET);
23         HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, SET);
24         break;
25     case STATE_YELLOW:
26         HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
27             SET);
28         HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
29             RESET);
30         HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, SET);
31         break;
32     case STATE_RED:
33         HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
34             SET);
35         HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
36             SET);
37         HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, RESET);
38         break;
39     }
40 }
41
42 /* USER CODE END 3 */

```

```

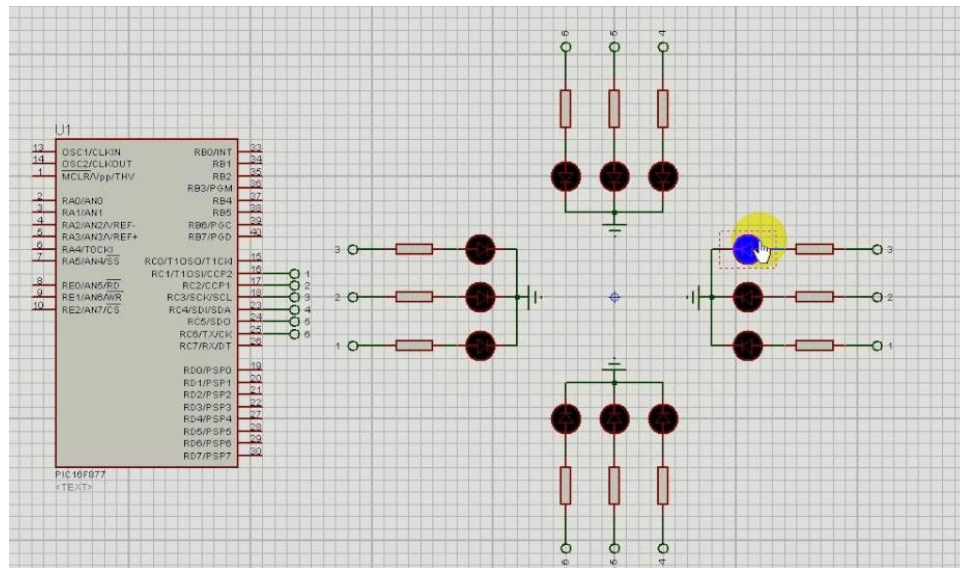
    SET);
33     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, RESET
    );
34     break;
35 //Error -> All LED light up
36 default:
37     HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
    RESET);
38     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
    RESET);
39     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, RESET
    );
40     break;
41 }
42 }
43 }
44
45 int GetNextState(int led_state){
46     switch(led_state){
47     case STATE_GREEN:
48         return STATE_YELLOW;
49
50     case STATE_YELLOW:
51         return STATE_RED;
52
53     case STATE_RED:
54         return STATE_GREEN;
55
56     default:
57         return -1;
58     }
59 }
60
61 int GetStateCounter(int led_state){
62     switch(led_state){
63     case STATE_GREEN:
64         return COUNTER_GREEN;
65
66     case STATE_YELLOW:
67         return COUNTER_YELLOW;
68
69     case STATE_RED:
70         return COUNTER_RED;
71
72     default:
73         return -1;
74     }
75 }

```

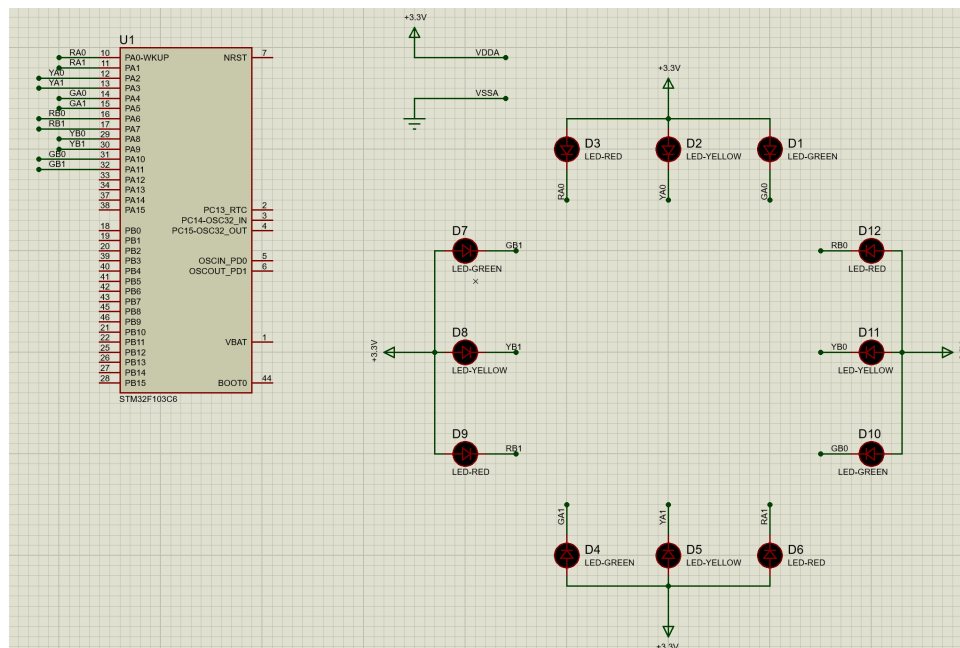
Program 1.4: Source code in the while loop and self-defined function's definition

4.3 Exercise 3

Extend to the 4-way traffic light. Arrange 12 LEDs in a nice shape to simulate the behaviors of a traffic light. A reference design can be found in the figure bellow.



Hình 1.26: Reference design for a 4 way traffic light



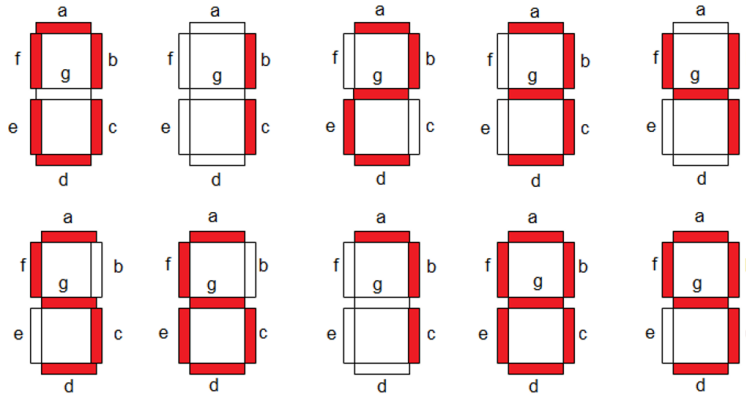
Hình 1.27: Schematic for exercise 3

4.4 Exercise 4

Add **only one 7 led segment** to the schematic in Exercise 3. This component can be found in Proteus by the keyword **7SEG-COM-ANODE**. For this device, the common pin should be connected to the power supply and other pins are supposed to

connected to PB0 to PB6. Therefore, to turn-on a segment in this 7SEG, the STM32 pin should be in logic 0 (0V).

Implement a function named **display7SEG(int num)**. The input for this function is from 0 to 9 and the outputs are listed as following:



Hình 1.28: Display a number on 7 segment LED

This function is invoked in the while loop for testing as following:

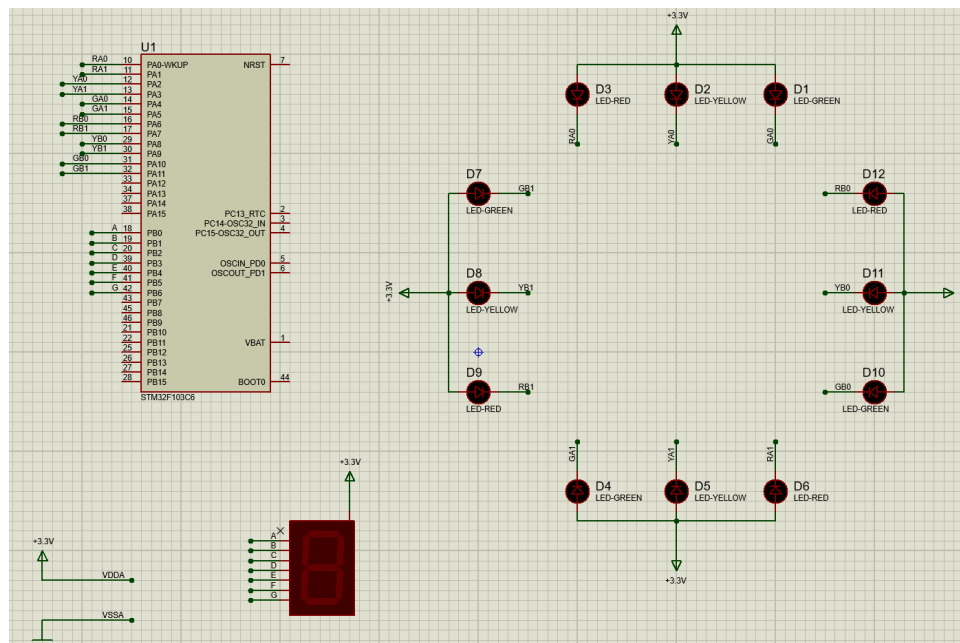
```

1 int counter = 0;
2 while (1){
3     if(counter >= 10) counter = 0;
4     display7SEG(counter++);
5     HAL_Delay(1000);
6 }
7

```

Program 1.5: An example for your source code

Report 1: Present the schematic.



Hình 1.29: Schematic for exercise 4

Report 2: Present the source code for display7SEG function.

```

1 void display7SEG(int num){
2     switch(num){
3         case 0:
4             HAL_GPIO_WritePin(SEG_A_GPIO_Port, SEG_A_Pin, RESET);
5             HAL_GPIO_WritePin(SEG_B_GPIO_Port, SEG_B_Pin, RESET);
6             HAL_GPIO_WritePin(SEG_C_GPIO_Port, SEG_C_Pin, RESET);
7             HAL_GPIO_WritePin(SEG_D_GPIO_Port, SEG_D_Pin, RESET);
8             HAL_GPIO_WritePin(SEG_E_GPIO_Port, SEG_E_Pin, RESET);
9             HAL_GPIO_WritePin(SEG_F_GPIO_Port, SEG_F_Pin, RESET);
10            HAL_GPIO_WritePin(SEG_G_GPIO_Port, SEG_G_Pin, SET);
11            break;
12
13        case 1:
14            HAL_GPIO_WritePin(SEG_A_GPIO_Port, SEG_A_Pin, SET);
15            HAL_GPIO_WritePin(SEG_B_GPIO_Port, SEG_B_Pin, RESET);

```

```

16     HAL_GPIO_WritePin(SEG_C_GPIO_Port, SEG_C_Pin, RESET);
17     HAL_GPIO_WritePin(SEG_D_GPIO_Port, SEG_D_Pin, SET);
18     HAL_GPIO_WritePin(SEG_E_GPIO_Port, SEG_E_Pin, SET);
19     HAL_GPIO_WritePin(SEG_F_GPIO_Port, SEG_F_Pin, SET);
20     HAL_GPIO_WritePin(SEG_G_GPIO_Port, SEG_G_Pin, SET);
21     break;
22
23 case 2:
24     HAL_GPIO_WritePin(SEG_A_GPIO_Port, SEG_A_Pin, RESET);
25     HAL_GPIO_WritePin(SEG_B_GPIO_Port, SEG_B_Pin, RESET);
26     HAL_GPIO_WritePin(SEG_C_GPIO_Port, SEG_C_Pin, SET);
27     HAL_GPIO_WritePin(SEG_D_GPIO_Port, SEG_D_Pin, RESET);
28     HAL_GPIO_WritePin(SEG_E_GPIO_Port, SEG_E_Pin, RESET);
29     HAL_GPIO_WritePin(SEG_F_GPIO_Port, SEG_F_Pin, SET);
30     HAL_GPIO_WritePin(SEG_G_GPIO_Port, SEG_G_Pin, RESET);
31     break;
32
33 case 3:
34     HAL_GPIO_WritePin(SEG_A_GPIO_Port, SEG_A_Pin, RESET);
35     HAL_GPIO_WritePin(SEG_B_GPIO_Port, SEG_B_Pin, RESET);
36     HAL_GPIO_WritePin(SEG_C_GPIO_Port, SEG_C_Pin, RESET);
37     HAL_GPIO_WritePin(SEG_D_GPIO_Port, SEG_D_Pin, RESET);
38     HAL_GPIO_WritePin(SEG_E_GPIO_Port, SEG_E_Pin, SET);
39     HAL_GPIO_WritePin(SEG_F_GPIO_Port, SEG_F_Pin, SET);
40     HAL_GPIO_WritePin(SEG_G_GPIO_Port, SEG_G_Pin, RESET);
41     break;
42
43 case 4:
44     HAL_GPIO_WritePin(SEG_A_GPIO_Port, SEG_A_Pin, SET);
45     HAL_GPIO_WritePin(SEG_B_GPIO_Port, SEG_B_Pin, RESET);
46     HAL_GPIO_WritePin(SEG_C_GPIO_Port, SEG_C_Pin, RESET);
47     HAL_GPIO_WritePin(SEG_D_GPIO_Port, SEG_D_Pin, SET);
48     HAL_GPIO_WritePin(SEG_E_GPIO_Port, SEG_E_Pin, SET);
49     HAL_GPIO_WritePin(SEG_F_GPIO_Port, SEG_F_Pin, RESET);
50     HAL_GPIO_WritePin(SEG_G_GPIO_Port, SEG_G_Pin, RESET);
51     break;
52
53 case 5:
54     HAL_GPIO_WritePin(SEG_A_GPIO_Port, SEG_A_Pin, RESET);
55     HAL_GPIO_WritePin(SEG_B_GPIO_Port, SEG_B_Pin, SET);
56     HAL_GPIO_WritePin(SEG_C_GPIO_Port, SEG_C_Pin, RESET);
57     HAL_GPIO_WritePin(SEG_D_GPIO_Port, SEG_D_Pin, RESET);
58     HAL_GPIO_WritePin(SEG_E_GPIO_Port, SEG_E_Pin, SET);
59     HAL_GPIO_WritePin(SEG_F_GPIO_Port, SEG_F_Pin, RESET);
60     HAL_GPIO_WritePin(SEG_G_GPIO_Port, SEG_G_Pin, RESET);
61     break;
62
63 case 6:
64     HAL_GPIO_WritePin(SEG_A_GPIO_Port, SEG_A_Pin, RESET);

```

```

65     HAL_GPIO_WritePin(SEG_B_GPIO_Port , SEG_B_Pin , SET);
66     HAL_GPIO_WritePin(SEG_C_GPIO_Port , SEG_C_Pin , RESET);
67     HAL_GPIO_WritePin(SEG_D_GPIO_Port , SEG_D_Pin , RESET);
68     HAL_GPIO_WritePin(SEG_E_GPIO_Port , SEG_E_Pin , RESET);
69     HAL_GPIO_WritePin(SEG_F_GPIO_Port , SEG_F_Pin , RESET);
70     HAL_GPIO_WritePin(SEG_G_GPIO_Port , SEG_G_Pin , RESET);
71     break;
72
73     case 7:
74         HAL_GPIO_WritePin(SEG_A_GPIO_Port , SEG_A_Pin , RESET);
75         HAL_GPIO_WritePin(SEG_B_GPIO_Port , SEG_B_Pin , RESET);
76         HAL_GPIO_WritePin(SEG_C_GPIO_Port , SEG_C_Pin , RESET);
77         HAL_GPIO_WritePin(SEG_D_GPIO_Port , SEG_D_Pin , SET);
78         HAL_GPIO_WritePin(SEG_E_GPIO_Port , SEG_E_Pin , SET);
79         HAL_GPIO_WritePin(SEG_F_GPIO_Port , SEG_F_Pin , SET);
80         HAL_GPIO_WritePin(SEG_G_GPIO_Port , SEG_G_Pin , SET);
81         break;
82
83     case 8:
84         HAL_GPIO_WritePin(SEG_A_GPIO_Port , SEG_A_Pin , RESET);
85         HAL_GPIO_WritePin(SEG_B_GPIO_Port , SEG_B_Pin , RESET);
86         HAL_GPIO_WritePin(SEG_C_GPIO_Port , SEG_C_Pin , RESET);
87         HAL_GPIO_WritePin(SEG_D_GPIO_Port , SEG_D_Pin , RESET);
88         HAL_GPIO_WritePin(SEG_E_GPIO_Port , SEG_E_Pin , RESET);
89         HAL_GPIO_WritePin(SEG_F_GPIO_Port , SEG_F_Pin , RESET);
90         HAL_GPIO_WritePin(SEG_G_GPIO_Port , SEG_G_Pin , RESET);
91         break;
92
93     case 9:
94         HAL_GPIO_WritePin(SEG_A_GPIO_Port , SEG_A_Pin , RESET);
95         HAL_GPIO_WritePin(SEG_B_GPIO_Port , SEG_B_Pin , RESET);
96         HAL_GPIO_WritePin(SEG_C_GPIO_Port , SEG_C_Pin , RESET);
97         HAL_GPIO_WritePin(SEG_D_GPIO_Port , SEG_D_Pin , RESET);
98         HAL_GPIO_WritePin(SEG_E_GPIO_Port , SEG_E_Pin , SET);
99         HAL_GPIO_WritePin(SEG_F_GPIO_Port , SEG_F_Pin , RESET);
100        HAL_GPIO_WritePin(SEG_G_GPIO_Port , SEG_G_Pin , RESET);
101        break;
102
103    default:
104        HAL_GPIO_WritePin(SEG_A_GPIO_Port , SEG_A_Pin , SET);
105        HAL_GPIO_WritePin(SEG_B_GPIO_Port , SEG_B_Pin , SET);
106        HAL_GPIO_WritePin(SEG_C_GPIO_Port , SEG_C_Pin , SET);
107        HAL_GPIO_WritePin(SEG_D_GPIO_Port , SEG_D_Pin , SET);
108        HAL_GPIO_WritePin(SEG_E_GPIO_Port , SEG_E_Pin , SET);
109        HAL_GPIO_WritePin(SEG_F_GPIO_Port , SEG_F_Pin , SET);
110        HAL_GPIO_WritePin(SEG_G_GPIO_Port , SEG_G_Pin , SET);
111        break;
112    }

```

Program 1.6: Source code for display7SEG

4.5 Exercise 5

Integrate the 7SEG-LED to the 4 way traffic light. In this case, the 7SEG-LED is used to display countdown value.

In this exercise, only source code is required to present. The function display7SEG in previous exercise can be re-used.

```

1      while (1)
2      {
3          /* USER CODE END WHILE */
4
5          /* USER CODE BEGIN 3 */
6
7          if(set_a_counter <= 0){
8              set_a_state = GetNextState(set_a_state);
9              set_a_counter = GetStateCounter(set_a_state);
10             DisplayLED(SET_A, set_a_state);
11         }
12
13         if(set_b_counter <= 0){
14             set_b_state = GetNextState(set_b_state);
15             set_b_counter = GetStateCounter(set_b_state);
16             DisplayLED(SET_B, set_b_state);
17         }
18
19         Display7SEG(SET_A, set_a_counter--);
20         Display7SEG(SET_B, set_b_counter--);
21
22         HAL_Delay(1000);
23     }
24     /* USER CODE END 3 */
25
26 int GetNextState(int led_state){
27     switch(led_state){
28     case STATE_GREEN:
29         return STATE_YELLOW;
30
31     case STATE_YELLOW:
32         return STATE_RED;
33
34     case STATE_RED:
35         return STATE_GREEN;
36
37     default:

```

```

38     return -1;
39 }
40 }
41
42 int GetStateCounter(int led_state){
43     switch(led_state){
44     case STATE_GREEN:
45         return COUNTER_GREEN;
46
47     case STATE_YELLOW:
48         return COUNTER_YELLOW;
49
50     case STATE_RED:
51         return COUNTER_RED;
52
53     default:
54         return -1;
55     }
56 }
57
58 void ResetLED(int led_set){
59     uint16_t start_pin = GetLEDSetPin(led_set);
60
61     if(start_pin == -1){
62         DisplayLEDError();
63         return;
64     }
65
66     HAL_GPIO_WritePin(GPIOA, start_pin << 0, SET);
67     HAL_GPIO_WritePin(GPIOA, start_pin << 1, SET);
68     HAL_GPIO_WritePin(GPIOA, start_pin << 2, SET);
69 }
70
71 void DisplayLED(int led_set, int led_state){
72     ResetLED(led_set);
73     uint16_t start_pin = GetLEDSetPin(led_set);
74
75     if(start_pin == -1){
76         DisplayLEDError();
77         return;
78     }
79
80     switch(led_state){
81     case STATE_GREEN:
82         HAL_GPIO_WritePin(GPIOA, start_pin << 2, RESET);
83         return;
84
85     case STATE_YELLOW:
86         HAL_GPIO_WritePin(GPIOA, start_pin << 1, RESET);

```

```

87     return;
88
89     case STATE_RED:
90         HAL_GPIO_WritePin(GPIOA, start_pin << 0, RESET);
91         return;
92
93     default:
94         DisplayLEDError();
95         return;
96 }
97 }
98
99 uint16_t GetLEDSetPin(int led_set){
100     switch(led_set){
101     case SET_A:
102         return LED_RED_0_Pin;
103
104     case SET_B:
105         return LED_RED_1_Pin;
106
107     //Catch error when led_set isn't SET_A or SET_B
108     default:
109         return -1;
110     }
111 }
112
113 void DisplayLEDError(){
114     //All LED turn on
115     HAL_GPIO_WritePin(GPIOA, LED_RED_0_Pin, RESET);
116     HAL_GPIO_WritePin(GPIOA, LED_YELLOW_0_Pin, RESET);
117     HAL_GPIO_WritePin(GPIOA, LED_GREEN_0_Pin, RESET);
118
119     HAL_GPIO_WritePin(GPIOA, LED_RED_1_Pin, RESET);
120     HAL_GPIO_WritePin(GPIOA, LED_YELLOW_1_Pin, RESET);
121     HAL_GPIO_WritePin(GPIOA, LED_GREEN_1_Pin, RESET);
122 }
123
124 void Display7SEG(int segment_set, int num){
125     Reset7SEG(segment_set);
126
127     uint16_t start_pin = Get7SEGSetPin(segment_set);
128     if(start_pin == -1){
129         Display7SEGEError();
130         return;
131     }
132
133     uint8_t number_code = GetNumberCode(num);
134     if(start_pin == -1){
135         Display7SEGEError();

```

```

136     return;
137 }
138
139 //CODE_SEG_x represent the on state of each segment
140 from g->a
141 //Example:
142 // CODE_SEG_A is 0_0000001 = 0x01
143 HAL_GPIO_WritePin(GPIOB, start_pin << 0, !(CODE_SEG_A &
144     number_code));
145 HAL_GPIO_WritePin(GPIOB, start_pin << 1, !(CODE_SEG_B &
146     number_code));
147 HAL_GPIO_WritePin(GPIOB, start_pin << 2, !(CODE_SEG_C &
148     number_code));
149 HAL_GPIO_WritePin(GPIOB, start_pin << 3, !(CODE_SEG_D &
150     number_code));
151 HAL_GPIO_WritePin(GPIOB, start_pin << 4, !(CODE_SEG_E &
152     number_code));
153 HAL_GPIO_WritePin(GPIOB, start_pin << 5, !(CODE_SEG_F &
154     number_code));
155 HAL_GPIO_WritePin(GPIOB, start_pin << 6, !(CODE_SEG_G &
156     number_code));
157 }
158
159 void Reset7SEG(int segment_set){
160     uint16_t start_pin = Get7SEGSetPin(segment_set);
161     if(start_pin == -1){
162         Display7SEGEError();
163         return;
164     }
165
166     HAL_GPIO_WritePin(GPIOB, start_pin << 0, SET);
167     HAL_GPIO_WritePin(GPIOB, start_pin << 1, SET);
168     HAL_GPIO_WritePin(GPIOB, start_pin << 2, SET);
169     HAL_GPIO_WritePin(GPIOB, start_pin << 3, SET);
170     HAL_GPIO_WritePin(GPIOB, start_pin << 4, SET);
171     HAL_GPIO_WritePin(GPIOB, start_pin << 5, SET);
172     HAL_GPIO_WritePin(GPIOB, start_pin << 6, SET);
173 }
174
175 uint16_t Get7SEGSetPin(int segment_set){
176     switch(segment_set){
177         case SET_A:
178             return SEG_A0_Pin;
179
180         case SET_B:
181             return SEG_A1_Pin;
182
183         //Catch error

```

```

177     default:
178         return -1;
179     }
180 }
181
182 void Display7SEGError(){
183     //If error happen, all of the segment with turn on
184     HAL_GPIO_WritePin(GPIOB, SEG_A0_Pin, SET);
185     HAL_GPIO_WritePin(GPIOB, SEG_B0_Pin, SET);
186     HAL_GPIO_WritePin(GPIOB, SEG_C0_Pin, SET);
187     HAL_GPIO_WritePin(GPIOB, SEG_D0_Pin, SET);
188     HAL_GPIO_WritePin(GPIOB, SEG_E0_Pin, SET);
189     HAL_GPIO_WritePin(GPIOB, SEG_F0_Pin, SET);
190     HAL_GPIO_WritePin(GPIOB, SEG_G0_Pin, RESET);
191
192     HAL_GPIO_WritePin(GPIOB, SEG_A1_Pin, SET);
193     HAL_GPIO_WritePin(GPIOB, SEG_B1_Pin, SET);
194     HAL_GPIO_WritePin(GPIOB, SEG_C1_Pin, SET);
195     HAL_GPIO_WritePin(GPIOB, SEG_D1_Pin, SET);
196     HAL_GPIO_WritePin(GPIOB, SEG_E1_Pin, SET);
197     HAL_GPIO_WritePin(GPIOB, SEG_F1_Pin, SET);
198     HAL_GPIO_WritePin(GPIOB, SEG_G1_Pin, RESET);
199 }
200
201 //Number is a set of 8 bits represent for each segment
202     going from g->a
203 //Example:
204 //  CODE_NUM_1: a=0, b=1, c=1, d=0, e=0, f=0, g=0
205 //  Therefore CODE_NUM_1 is 0_0000110 = 0x06
206 uint8_t GetNumberCode(int number){
207     switch(number){
208     case 0:
209         return CODE_NUM_0;
210
211     case 1:
212         return CODE_NUM_1;
213
214     case 2:
215         return CODE_NUM_2;
216
217     case 3:
218         return CODE_NUM_3;
219
220     case 4:
221         return CODE_NUM_4;
222
223     case 5:
224         return CODE_NUM_5;

```

```

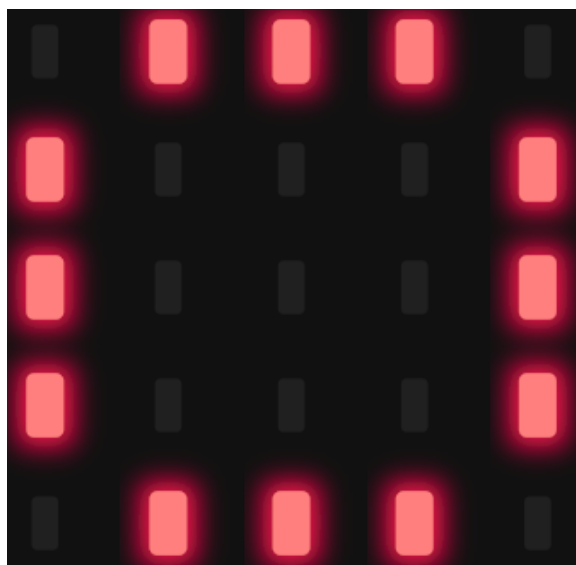
225     case 6:
226         return CODE_NUM_6;
227
228     case 7:
229         return CODE_NUM_7;
230
231     case 8:
232         return CODE_NUM_8;
233
234     case 9:
235         return CODE_NUM_9;
236
237     default:
238         return -1;
239 }
240 }

```

Program 1.7: Source code in the while loop and self-defined function definition

4.6 Exercise 6

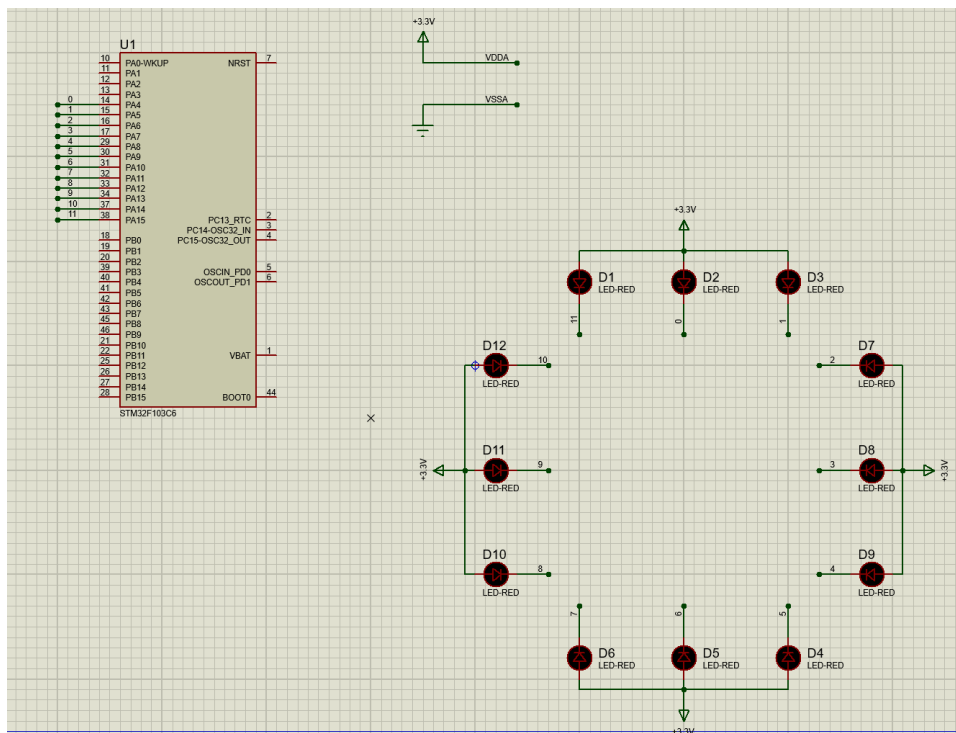
In this exercise, a new Proteus schematic is designed to simulate an analog clock, with 12 different number. The connections for 12 LEDs are supposed from PA4 to PA15 of the STM32. The arrangement of 12 LEDs is depicted as follows.



Hình 1.30: 12 LEDs for an analog clock

Report 1: Present the schematic.

Report 2: Implement a simple program to test the connection of every single LED. This testing program should turn every LED in a sequence.



Hình 1.31: Schematic for exercise 6

```

1 while (1)
2 {
3     /* USER CODE END WHILE */
4
5     /* USER CODE BEGIN 3 */
6     HAL_GPIO_TogglePin(GPIOA, LED_0_Pin << counter);
7     counter++;
8     if(counter >= 12){
9         counter = 0;
10    }
11
12    HAL_Delay(1000);
13 }

```

Program 1.8: Simple testing program

4.7 Exercise 7

Implement a function named **clearAllClock()** to turn off all 12 LEDs. Present the source code of this function.

```

1 void clearAllClock(){
2     HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 0, SET);
3     HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 1, SET);
4     HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 2, SET);
5     HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 3, SET);
6     HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 4, SET);

```

```

7  HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 5, SET);
8  HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 6, SET);
9  HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 7, SET);
10 HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 8, SET);
11 HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 9, SET);
12 HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 10, SET);
13 HAL_GPIO_WritePin(GPIOA, LED_0_Pin << 11, SET);
14 }

```

Program 1.9: Function Implementation

4.8 Exercise 8

Implement a function named **setNumberOnClock(int num)**. The input for this function is from **0 to 11** and an appropriate LED is turn on. Present the source code of this function.

```

1 void setNumberOnClock(int number){
2     HAL_GPIO_WritePin(GPIOA, LED_0_Pin << number, RESET);
3 }

```

Program 1.10: Function Implementation

4.9 Exercise 9

Implement a function named **clearNumberOnClock(int num)**. The input for this function is from **0 to 11** and an appropriate LED is turn off.

```

1 void clearNumberOnClock(int number){
2     HAL_GPIO_WritePin(GPIOA, LED_0_Pin << number, SET);
3 }

```

Program 1.11: Function Implementation

4.10 Exercise 10

Integrate the whole system and use 12 LEDs to display a clock. At a given time, there are only 3 LEDs are turn on for hour, minute and second information.

```

1 while (1)
2 {
3     /* USER CODE END WHILE */
4
5     /* USER CODE BEGIN 3 */
6     clearAllClock();
7     setNumberOnClock(second);
8     setNumberOnClock(minute);
9     setNumberOnClock(hour);
10

```

```

11 //Update counter
12   sec_counter--;
13   //Update every 5sec
14   if(sec_counter <= 0){
15       second++;
16       sec_counter = 5;
17   }
18
19   //Reset every 1min -> update min_counter
20   if(second >= 12){
21       min_counter--;
22       second = 0;
23   }
24
25   //Update every 5min
26   if(min_counter <= 0){
27       minute++;
28       min_counter = 5;
29   }
30
31   //Update hour every 12 min_LED, reset minute
32   if(minute >= 12){
33       hour++;
34       minute = 0;
35   }
36
37   //Reset after 12hour
38   if(hour >= 12){
39       hour = 0;
40   }
41
42   HAL_Delay(1000);
43   }

```

Program 1.12: Function Implementation

Google Drive Link for all of the smulation videos:

<https://drive.google.com/drive/folders/1C22HR1m3bZEKceJpj8bn702kxfe33Zx5?usp=sharing>